P.PORTO

ESCOLA
SUPERIOR
DE TECNOLOGIA
E GESTÃO

# Relatório

## Programação em Ambiente Web

2025/2026


Relatório elaborado por:

**Grupo 25**

8240344 Mariana Silva Barreto

8240334 Joana Francisca Pacheco Rodrigues

8240303 José Carlos Campanhã Dias

Curso de:

Licenciatura em Engenharia Informática

Docentes:

Fábio Silva (fas@estg.ipp.pt)

João Ramos (jrmr@estg.ipp.pt)

Nelson Rodrigues (njr@estg.ipp.pt)


Felgueiras, 28 de maio de 2026

# Conteúdo


**1 Identificação e Caracterização do Projeto** **3**

1.1 Contexto e Objetivos do Negócio . . . . . 3

1.2 Abordagem ao Processo de Negócio . . . . . 4

1.3 Planeamento e Desenvolvimento . . . . . 5

1.4 Contas de Teste (Credenciais de Acesso) . . . . . 7

**2 Especificação Geral do Software** **8**

2.1 Arquitetura e Tecnologias Base . . . . . 8

2.2 Modelação de Dados . . . . . 9

2.3 Mapeamento de Rotas Desenvolvidas . . . . . 11

2.4 Mapeamento dos Requisitos de Negócio com a Implementação . . 13

**3 Análise e Execução** **14**

3.1 Apreciação Crítica do Plano Inicial . . . . . 14

3.2 Regras de Negócio e Lógica de Estados . . . . . 16

3.3 Venda em Caixa e Integração EAN . . . . . 17

**4 Frontoffice e API Rest** **18**

4.1 Evolução da Arquitetura para o Milestone 2 . . . . . 18

4.2 Funcionalidades do Frontoffice Angular . . . . . 19

4.2.1 Dashboards por Perfil . . . . . 20

4.3 Rotas do Frontoffice Angular . . . . . 21

4.4 API REST e Documentação Swagger . . . . . 21

4.5 Segurança e Autenticação . . . . . 23

4.6 Funcionalidades de Bonificação Implementadas . . . . . 24

4.6.1 Funcionalidades de Bonificação Implementadas . . . . . 24

4.7 Interface Visual do Sistema . . . . . 25

**5 Execução do Projeto** **26**

**6 Conclusão e Trabalho Futuro** **28**

6.1 Síntese dos Resultados Alcançados . . . . . 28

6.2 Trabalho Futuro . . . . . 29

# Identificação e Caracterização do Projeto

## 1.1 Contexto e Objetivos do Negócio

Este projeto consiste no desenvolvimento de uma plataforma digital, composta por um *Marketplace* e um *Backoffice*, dedicada à dinamização de supermercados locais. O objetivo central é capacitar estes estabelecimentos com ferramentas tecnológicas que permitam a gestão de produtos e encomendas, facilitando a expansão da sua base de clientes através de um sistema de entregas realizado por estafetas.

Nesta primeira fase (*Milestone* 1), o foco incidiu na construção do *Backoffice* e do sistema de venda. Foi implementada uma gestão de catálogo de produtos, permitindo a organização por categorias, a definição de preços e o controlo de *stock* disponível. Esta componente assegura a consistência de dados entre as vendas presenciais e o inventário do sistema, garantindo que a disponibilidade de produtos é atualizada em tempo real.

## 1.2 Abordagem ao Processo de Negócio

A arquitetura da solução separa as operações através de quatro perfis de utilizador (*roles*), cada um com permissões e fluxos específicos:

- **Administrador** (`admin`): Responsável pela moderação (aprovação de lojas), gestão de utilizadores, categorias e cupões.
- **Supermercado** (`supermercado`): Entidade comercial que gere produtos, stock (com suporte EAN), vendas em caixa e encomendas.
- **Estafeta** (`estafeta`): Operador logístico que aceita entregas e gere os respetivos estados.
- **Cliente** (`cliente`): Utilizador final que consulta produtos, histórico e submete avaliações.

O fluxo de negócio da plataforma pode ser descrito da seguinte forma: um supermercado regista-se na plataforma e aguarda aprovação do administrador; após aprovação, pode configurar o seu catálogo de produtos, definir métodos de entrega e custos associados. O cliente, por sua vez, regista-se de forma autónoma no frontoffice Angular, pesquisa e compara produtos entre supermercados, adiciona artigos ao carrinho e finaliza a encomenda com pagamento via Stripe. Após confirmação do pagamento, o supermercado recebe a encomenda, processa-a e, caso o método de entrega seja por estafeta, a encomenda fica disponível para aceitação. O estafeta aceita a entrega, atualiza o estado até à conclusão, e o cliente pode então submeter uma avaliação do supermercado e do estafeta. Em paralelo, o sistema envia notificações automáticas por email ao cliente em cada transição de estado relevante.

## 1.3 Planeamento e Desenvolvimento

Para garantir o cumprimento do **Milestone 1**, o trabalho foi dividido em fases, permitindo que cada módulo fosse desenvolvido, testado e validado progressivamente antes da integração final.

O desenvolvimento foi organizado em etapas, iniciando pela configuração da arquitetura base da aplicação e persistência de dados, seguindo-se a autenticação e controlo de permissões, desenvolvimento do painel administrativo, gestão de supermercados e produtos, sistema de vendas em caixa e, por fim, integrações externas como notificações por email e pagamentos online.

A Tabela 1.1 apresenta a distribuição das principais tarefas, os elementos responsáveis e o tempo previsto para cada fase.

Tabela 1.1: Plano de Desenvolvimento — Milestone 1

| **Tarefa** | **Descrição** | **Responsável** | **Tempo** |
|---|---|---|---|
| Setup & DB | Configuração Express e MongoDB | Joana / Mariana | 10h |
| Auth & Roles | Registo/Login e Permissões | Todos | 10h |
| Admin Panel | Gestão de lojas e categorias | Todos | 12h |
| Lojas & Prod. | CRUD, Imagens e EAN | Joana / José | 15h |
| Checkout | Encomendas e Stripe | Joana / José | 20h |
| Notificações | Nodemailer e Mailgen | Todos | 10h |

Esta abordagem permitiu reduzir erros de integração, facilitar testes progressivos e garantir maior controlo sobre o cumprimento dos requisitos definidos para o Milestone 1. De referir que algumas tarefas, nomeadamente o Checkout e a lógica de encomendas, ultrapassaram o tempo inicialmente previsto devido à complexidade da integração com a Stripe e à necessidade de garantir consistência de dados em cenários de falha.

A segunda fase **(Milestone 2)** separou o sistema em duas partes: o backend foi transformado numa API REST documentada, e o Frontoffice foi desenvolvido de raiz em Angular.

A Tabela 1.2 apresenta a distribuição das principais tarefas desta fase, identificando os elementos responsáveis e a estimativa de tempo dedicado a cada bloco de trabalho:

Tabela 1.2: Plano de Desenvolvimento – Milestone 2

| **Tarefa** | **Descrição** | **Responsável** | **Tempo** |
|---|---|---|---|
| API & Auth | API REST, JWT, Swagger, Stripe Checkout e estados | Joana / Mariana | 18h |
| Angular | Base Angular, autenticação, pesquisa, filtros e comparação | Mariana / Joana | 20h |
| Área Cliente | Carrinho, checkout, histórico, detalhe, reviews e dashboard | José | 22h |
| Validação | Integração final, testes e validação | Todos | 12h |

## 1.4 Contas de Teste (Credenciais de Acesso)

Para validação, a base de dados dispõe das seguintes credenciais:

Tabela 1.3: Logins de Teste para Validação

| **Perfil** | **Email** | **Password** | **Observação** |
|---|---|---|---|
| Admin | admin@ad.pt | admin | Moderação total |
| Supermercado | supemercado@super.pt | supermercado | Loja aprovada |
| Estafeta | estafeta@est.pt | estafeta | Logística |
| Cliente | cliente@cli.pt | cliente | Histórico/Avaliação |
| Cliente | josecarlosdias0009@gmail.com | jose | Teste Frontoffice |

# Especificação Geral do Software

## 2.1 Arquitetura e Tecnologias Base

A aplicação adota o padrão MVC (Model-View-Controller), utilizando Node.js e a framework Express.js. A persistência é assegurada pelo MongoDB Atlas via Mongoose, enquanto a interface utiliza EJS e Bootstrap. Foram ainda integrados serviços de autenticação por sessões, encriptação com Bcrypt e pagamentos via Stripe.

A escolha destas tecnologias baseou-se na adequação aos requisitos do projeto e na facilidade de manutenção da solução:

- **Express.js**: Permite uma estrutura modular, facilitando a implementação do padrão MVC exigido.
- **MongoDB Atlas**: Oferece um armazenamento flexível de documentos, ideal para a evolução de entidades como encomendas e produtos sem migrações complexas. A utilização da versão cloud (Atlas) eliminou a necessidade de infraestrutura local, facilitou o desenvolvimento colaborativo entre os membros do grupo e garantiu disponibilidade contínua com backups automáticos.
- **EJS**: Utilizado como *template engine* por permitir a renderização dinâmica no servidor e simplicidade na criação do backoffice.
- **Stripe Checkout**: Escolhido para simular pagamentos reais de forma segura e com integração direta no fluxo de encomendas.
- **Nodemailer + Mailgen**: Responsáveis pelo envio de notificações automáticas, aproximando a aplicação de um cenário real de produção.
- **Sessions**: Privilegiou-se o uso de sessões (em detrimento de JWT) pela simplicidade e melhor controlo da autenticação *server-side* nesta fase, re-

servando o JWT exclusivamente para a API REST consumida pelo frontoffice Angular.

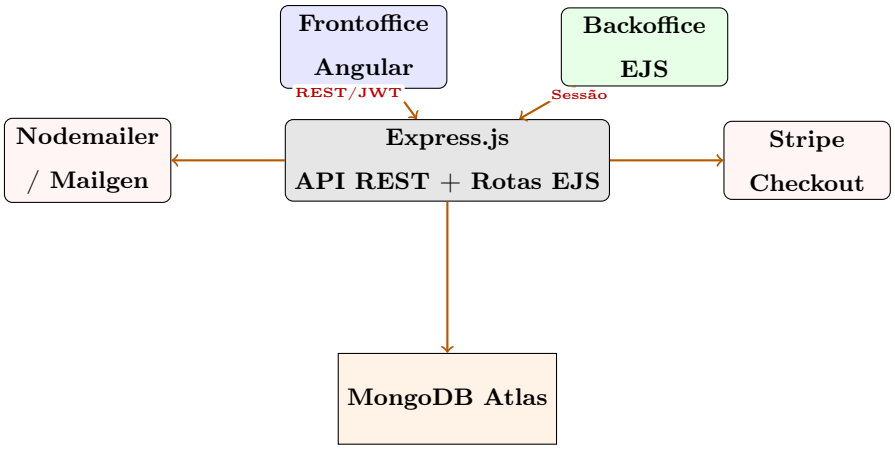



Figura 2.1: Diagrama de arquitetura do sistema.

**Tecnologias Utilizadas — Milestone 1**

**Runtime/DB:** Node.js · Express.js · MongoDB Atlas · Mongoose

**Frontend:** EJS · Bootstrap · Mailgen

**Integrações:** Stripe Checkout · Nodemailer · Multer · Bcrypt

## 2.2 Modelação de Dados

A estrutura de dados consiste em seis coleções principais:

1. `User` (Perfis e Roles);
2. `Supermarket` (Aprovação e entrega);
3. `Product` (*Stock*, EAN e imagens);
4. `Category` (Organização de catálogo);
5. `Order` (Estados de encomenda e itens);
6. `Coupon` (Cupões de desconto).

A estrutura de dados da aplicação foi desenhada em torno de seis coleções principais no MongoDB Atlas, organizadas de forma a refletir diretamente as entidades do negócio e as suas relações.

- **User** — Centraliza todos os perfis do sistema (*admin*, supermercado, estafeta, cliente) através de um campo `role`. Armazena dados de autenticação (email, password encriptada com Bcrypt), dados pessoais (nome, morada, telefone) e, no caso do perfil cliente, o histórico de encomendas associado por referência.
- **Supermarket** — Representa cada loja registada na plataforma. Contém nome, descrição, localização (usada para geocodificação via Nominatim), horário de funcionamento, métodos de entrega disponíveis, custo de entrega por método, e o campo `isApproved` que controla se a loja pode ou não operar. Está associado ao `User` com *role* de supermercado através de uma referência.
- **Product** — Cada produto pertence a um único supermercado (referência a `Supermarket`), e inclui nome, descrição, categoria, preço, imagem (gerida via Multer), stock disponível, código EAN e o campo `isActive` que permite desativar um produto sem o eliminar da base de dados.
- **Category** — Coleção simples gerida pelo administrador, usada para organizar o catálogo de produtos. Cada produto referencia uma categoria.
- **Order** — A coleção mais complexa do sistema. Inclui referências ao cliente, ao supermercado e ao estafeta (quando aplicável), a lista de itens com quantidade e preço unitário no momento da compra, o valor total, o método de entrega, o custo de entrega, o estado atual (pendente, confirmada, em preparação, em entrega, entregue, cancelada), a referência ao pagamento Stripe e os campos de avaliação após entrega.
- **Coupon** — Armazena cupões de desconto criados pelo administrador, com código, percentagem de desconto, data de validade e número máximo de utilizações. É validado e contabilizado no momento da criação da encomenda.

As relações entre coleções são implementadas através de referências (`ObjectId`) com população via Mongoose, privilegiando a flexibilidade do modelo de documentos para a evolução futura das entidades.

## 2.3 Mapeamento de Rotas Desenvolvidas

Esta tabela apresenta o núcleo do sistema de rotas que estruturam a lógica de negócio:

Tabela 2.1: Mapeamento Completo de Rotas e Funcionalidades

| **Rota** | **Acesso** | **Funcionalidade** |
|---|---|---|
| /users/register | Público | Registo de novos utilizadores. |
| /users/login | Público | Autenticação e redirecionamento por perfil. |
| /users/logout | Autenticado | Terminar sessão em segurança. |
| /admin/dashboard | Admin | Gestão de utilizadores e aprovação de lojas. |
| /admin/categories | Admin | Gestão de categorias do Marketplace. |
| /admin/coupons | Admin | Criação e controlo de cupões de desconto. |
| /supermarket/dashboard | Supermercado | Estatísticas de vendas e produtos populares. |
| /supermarket/new | Supermercado | Registo de nova unidade de venda. |
| /supermarket/:id/products | Supermercado | Gestão de inventário e CRUD de produtos. |
| /orders/create/:id | Superm./Admin | Interface Venda em Caixa. |
| /orders | Autenticado | Listagem/Histórico de encomendas por perfil. |
| /orders/:id | Autenticado | Detalhe completo da encomenda e itens. |
| /orders/:id/claim | Estafeta | Aceitação de entrega disponível. |
| /orders/:id/status | Autenticado | Atualização do estado da encomenda |
| /courier/dashboard | Estafeta | Gestão de entregas e disponibilidade. |
| /payments/checkout/:id | Cliente | Início do fluxo de pagamento Stripe. |
| /payments/webhook | Sistema | Validação de pagamentos concluídos. |

Tabela 2.2: Endpoints REST principais consumidos pelo Angular

| **Rota** | **Método** | **Objetivo** |
|---|---|---|
| /api/auth/register, /api/auth/login, /api/auth/me | POST / GET | Autenticação e perfil do cliente |
| /api/products, /api/products/compare | GET | Pesquisa, filtros e comparação de preços |
| /api/categories, /api/supermarkets | GET | Catálogo auxiliar e escolha de supermercado |
| /api/supermarkets/:id/reviews | GET | Reviews públicas de uma loja |
| /api/orders, /api/orders/my, /api/orders/:id | POST / GET | Criação e consulta de encomendas |
| /api/orders/:id/cancel | POST | Cancelamento até 5 min após confirmação |
| /api/orders/:id/rate | POST | Submissão de review (supermercado e estafeta) |
| /api/dashboard/client | GET | Estatísticas do cliente |
| /api/payments/checkout/:id | POST | Arranque do pagamento Stripe |
| /api/payments/confirm/:id | GET | Confirmação do pagamento |

## 2.4 Mapeamento dos Requisitos de Negócio com a Implementação

De forma a garantir que os requisitos definidos no enunciado fossem corretamente traduzidos em funcionalidades concretas, foi realizado o seguinte mapeamento entre necessidades do negócio e componentes de software desenvolvidas.

Tabela 2.3: Mapeamento entre Requisitos de Negócio e Implementação

| **Requisito de Negócio** | **Solução Implementada** |
|---|---|
| Aprovação de supermercados | Dashboard Admin com aprovação/rejeição |
| Gestão de produtos e stock | CRUD de produtos + validação de stock |
| Venda presencial em loja | Interface de Venda em Caixa em `/orders/create/:id` |
| Entrega por estafeta | Dashboard de estafeta + aceitação de entregas |
| Notificações ao cliente | Sistema de emails automáticos |
| Pagamento online | Integração Stripe Checkout |
| Cupões de desconto | Gestão de cupões no painel Admin |
| Avaliação pós-entrega | Sistema de ratings para cliente |
| Desativação de produtos sem apagamento | Campo isActive e ação de ativar/desativar no painel |
| Frontoffice cliente em Angular | Aplicação Angular com catálogo, comparação, carrinho, checkout, histórico, dashboard e perfil |
| Pesquisa e filtros de produtos | Filtros por supermercado, categoria, texto e ordenação por preço |
| Cancelamento pelo cliente | Endpoint REST dedicado com regra temporal de 5 minutos |
| Reviews de supermercados e estafetas | Dono do Supermercado consulta todas as reviews; cliente consulta reviews por supermercado |

# Análise e Execução

## 3.1 Apreciação Crítica do Plano Inicial

Durante a fase inicial do projeto, o objetivo principal consistia em cumprir os requisitos obrigatórios definidos para o Milestone 1, nomeadamente o desenvolvimento do backoffice, parametrização de supermercados, gestão de produtos e registo de vendas em caixa.

Ao longo do desenvolvimento foi possível expandir significativamente o âmbito do trabalho, sendo implementadas várias funcionalidades adicionais de bonificação: sistema de cupões, notificações automáticas por email, integração com Stripe Checkout, dashboards com estatísticas, avaliações de supermercados e estafetas, e suporte para leitura de código de barras EAN.

As principais dificuldades técnicas centraram-se em quatro áreas. A primeira foi o controlo rigoroso dos estados das encomendas e respetivas transições, uma vez que qualquer inconsistência comprometia a integridade dos dados entre o stock, os pagamentos e os estafetas. A segunda foi a integração dos *webhooks* da Stripe, que exigiu a configuração de um *endpoint* dedicado e a validação criptográfica da assinatura de cada evento, sendo que em ambiente de desenvolvimento foi necessário recorrer ao Stripe CLI para simular eventos localmente. A terceira dificuldade foi a gestão de permissões para os quatro perfis distintos, especialmente nos casos em que uma mesma rota devia comportar-se de forma diferente consoante o *role* do utilizador autenticado. Por fim, a coexistência de dois mecanismos de autenticação — sessões no backoffice EJS e JWT no frontoffice Angular — introduziu complexidade adicional na gestão do estado de autenticação, exigindo *middlewares* distintos e uma separação clara das rotas da API.

Relativamente à opção de usar sessões no backoffice em detrimento de JWT, considera-se que foi a escolha mais adequada para esta fase: o backoffice é uma aplicação *server-side* com renderização EJS, onde o controlo de sessão no servidor é mais simples e seguro. O JWT foi reservado para a API REST consumida pelo Angular, garantindo uma camada *stateless* e escalável para o frontoffice.

Algumas tarefas ultrapassaram o tempo inicialmente previsto, em particular a lógica de encomendas e pagamentos, devido à necessidade de garantir consistência de dados em cenários de falha (por exemplo, cancelamento de pagamento com reposição de stock). Apesar disso, o resultado final superou o planeamento inicial, resultando num sistema funcional que cobre todos os requisitos obrigatórios e a maioria das funcionalidades de bonificação.

## 3.2 Regras de Negócio e Lógica de Estados

O sistema garante a integridade das operações através de validações rigorosas em cada transição de estado da encomenda. A lógica de negócio foi programada para ser linear, impedindo transições inválidas (ex.: uma encomenda não pode passar de "pendente"diretamente para "em entrega"). A Tabela 3.1 detalha a conformidade das principais regras implementadas nesta fase:

Tabela 3.1: Conformidade das Regras de Negócio

| **Regra de Negócio** | **Estado** |
|---|---|
| Supermercado só pode vender após aprovação do Admin | Implementado |
| Validação de stock (impede stock negativo) | Implementado |
| Reposição automática de stock em caso de cancelamento | Implementado |
| Um estafeta só pode aceitar uma entrega de cada vez | Implementado |
| Cliente é validado/criado automaticamente na venda em caixa | Implementado |
| Cupão é validado e contabilizado antes da encomenda | Implementado |
| Pagamento Stripe exige confirmação via Webhook/Redirecionamento | Implementado |
| Reviews apenas após encomenda entregue | Implementado |

A figura seguinte representa os diferentes estados de uma Encomenda:

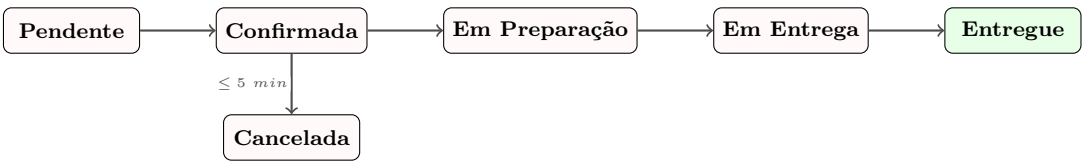



Figura 3.1: Diagrama de Estados da Encomenda.

## 3.3 Venda em Caixa e Integração EAN

A interface de "Venda em Caixa"foi desenhada para simular o processo de venda presencial em supermercados locais. Esta componente comunica em tempo real com o "cliente", garantindo a atualização imediata dos níveis de *stock* após cada transação.

**Nota: Leitura de Código de Barras**

A interface Venda em Caixa suporta a leitura de códigos EAN. Ao inserir ou digitalizar o código no formulário de venda, o sistema identifica automaticamente o produto correspondente através de uma pesquisa na base de dados, atualizando a quantidade no carrinho de forma automática. Esta funcionalidade simula a eficiência de um terminal de vendas profissional e minimiza erros de introdução manual.

# Frontoffice e API Rest

## 4.1 Evolução da Arquitetura para o Milestone 2

O Milestone 2 introduziu uma separação clara entre a camada de apresentação e a lógica de negócio. O backend Express.js foi refatorizado para atuar exclusivamente como fornecedor de serviços REST, enquanto o frontoffice do cliente foi desenvolvido de raiz em Angular, comunicando assincronamente com o servidor através de pedidos HTTP com troca de dados em formato JSON.

A autenticação no frontoffice é realizada via **JWT** (*JSON Web Tokens*), distinta da autenticação por sessão utilizada no backoffice EJS, permitindo uma API stateless e segura.

## 4.2 Funcionalidades do Frontoffice Angular

A aplicação Angular foi desenhada exclusivamente para os clientes finais, respondendo diretamente aos requisitos de negócio estabelecidos:

- **Catálogo e Pesquisa:** Listagem de produtos com filtros combinados por texto, categoria, supermercado e ordenação por preço.
- **Comparação de Preços:** Visualização do mesmo produto vendido por diferentes supermercados, permitindo ao cliente escolher a opção mais económica.
- **Carrinho de Compras:** Adição, alteração de quantidade e remoção de produtos, com validação de stock em tempo real.
- **Checkout:** Seleção de método de entrega (levantamento em loja ou estafeta), aplicação de cupões de desconto e cálculo automático do total.
- **Pagamento via Stripe:** Integração com a API Stripe Checkout para simulação de pagamentos seguros, com confirmação por redireccionamento.
- **Histórico de Encomendas:** Consulta do estado em tempo real de todas as encomendas, com possibilidade de cancelamento nos primeiros 5 minutos.
- **Reviews:** Submissão de avaliações de supermercados e estafetas após entrega confirmada. Consulta pública de reviews por supermercado e por estafeta.
- **Dashboard do Cliente:** Visão agregada com total gasto, produtos mais comprados e encomendas recentes.
- **Perfil:** Consulta e atualização dos dados pessoais do cliente.
- **Mapas de Geolocalização:** Visualização da localização de cada supermercado através de mapas interativos (Leaflet + OpenStreetMap).

### 4.2.1 Dashboards por Perfil

O sistema implementa dashboards distintos para cada perfil de utilizador, apresentando informação relevante e contextualizada para cada função na plataforma.

- **Dashboard do Cliente**: Disponível no frontoffice Angular, apresenta o total gasto na plataforma, os produtos mais comprados e as encomendas mais recentes com o respetivo estado atual.
- **Dashboard do Supermercado**: Acessível no backoffice EJS, apresenta o número total de encomendas recebidas, os produtos mais vendidos por volume, o valor total de vendas e as encomendas pendentes que requerem ação.
- **Dashboard do Estafeta**: Disponível no backoffice EJS, apresenta o número total de entregas realizadas, os supermercados com maior número de entregas associadas e as entregas atualmente em curso ou disponíveis para aceitação.
- **Dashboard do Administrador**: Acessível no backoffice EJS, apresenta uma visão global da plataforma: número total de utilizadores registados por perfil, supermercados ativos e pendentes de aprovação, total de encomendas na plataforma e cupões em vigor.

## 4.3 Rotas do Frontoffice Angular

Tabela 4.1: Rotas da Aplicação Angular

| **Rota** | **Funcionalidade** |
|---|---|
| /products | Catálogo com filtros por texto, categoria e supermercado |
| /compare | Comparação de preços do mesmo produto entre lojas |
| /supermarkets | Listagem de supermercados aprovados |
| /supermarkets/:id | Detalhe do supermercado, produtos, reviews e estafetas |
| /supermarkets/:id/reviews | Reviews detalhadas de um supermercado |
| /couriers/:id/reviews | Reviews detalhadas de um estafeta |
| /cart | Carrinho de compras |
| /checkout | Finalização da compra e escolha de entrega |
| /orders | Histórico de encomendas |
| /orders/:id | Detalhe, cancelamento, pagamento e reviews da encomenda |
| /dashboard | Estatísticas do cliente |
| /profile | Dados pessoais |
| /login | Autenticação JWT |
| /register | Registo de novo cliente |

## 4.4 API REST e Documentação Swagger

Toda a camada de serviços REST foi documentada com **Swagger** (OpenAPI), acessível em `/api-docs`. A documentação cobre todos os endpoints consumidos pelo Angular, organizados por módulos de negócio: autenticação, produtos, supermercados, encomendas e pagamentos.

Tabela 4.2: Endpoints REST da API

| **Endpoint** | **Método** | **Objetivo** |
|---|---|---|
| /api/auth/register, /api/auth/login | POST | Registo e autenticação |
| /api/auth/me | GET | Perfil do cliente autenticado |
| /api/products | GET | Catálogo com filtros e paginação |
| /api/products/compare | GET | Comparação de preços |
| /api/categories | GET | Lista de categorias |
| /api/supermarkets | GET | Supermercados aprovados |
| /api/supermarkets/:id/reviews | GET | Reviews de um supermercado |
| /api/supermarkets/:id/couriers | GET | Estafetas e reviews por supermercado |
| /api/couriers/:id/reviews | GET | Reviews de um estafeta |
| /api/orders | POST | Criação de encomenda |
| /api/orders/my | GET | Encomendas do cliente |
| /api/orders/:id | GET | Detalhe de encomenda |
| /api/orders/:id/cancel | POST | Cancelamento (janela de 5 min) |
| /api/orders/:id/rate | POST | Submissão de review |
| /api/orders/:id/refund-request | POST | Pedido de reembolso |
| /api/dashboard/client | GET | Estatísticas do cliente |
| /api/payments/checkout/:id | POST | Início do pagamento Stripe |
| /api/payments/confirm/:id | GET | Confirmação do pagamento |

## 4.5 Segurança e Autenticação

A API REST utiliza JWT para autenticação *stateless*. Cada pedido autenticado inclui o token no cabeçalho `Authorization: Bearer <token>`. O *middleware* `requireJWT` valida o token em todas as rotas protegidas, e o *middleware* `requireClientJWT` restringe o acesso exclusivamente a utilizadores com o perfil de cliente, impedindo que outros perfis acedam ao frontoffice Angular.

Para além da autenticação por token, a API implementa CORS (*Cross-Origin Resource Sharing*) de forma restritiva, permitindo apenas pedidos provenientes da origem do frontoffice Angular (`http://localhost:4200` em desenvolvimento). Esta configuração é essencial numa arquitetura onde o frontend e o backend correm em portas distintas, prevenindo pedidos cross-origin não autorizados por parte de outras origens.

As passwords de todos os utilizadores são armazenadas de forma segura através de *hashing* com Bcrypt, nunca sendo persistidas em texto simples. A chave de assinatura JWT e as credenciais Stripe são mantidas em variáveis de ambiente através do ficheiro `.env`, nunca expostas no código fonte.

## 4.6 Funcionalidades de Bonificação Implementadas

### 4.6.1 Funcionalidades de Bonificação Implementadas

Para além dos requisitos base, foram implementadas todas as funcionalidades de bonificação propostas no enunciado, bem como uma funcionalidade adicional desenvolvida por iniciativa própria do grupo:

- **Notificações por Email**: Envio automático de emails ao cliente em cada transição de estado da encomenda, utilizando Nodemailer e Mailgen para geração de templates HTML.
- **Pagamento Online**: Integração com a API Stripe Checkout para simulação de pagamentos seguros, com confirmação por redirecionamento e validação via *webhook*.
- **Sistema de Avaliações**: Submissão de avaliações de supermercados e estafetas após entrega confirmada, com consulta pública de *reviews* por supermercado e por estafeta.
- **Cupões de Desconto**: Gestão de cupões no painel do administrador, com validação de código, percentagem de desconto, data de validade e limite de utilizações, aplicados no momento do checkout.
- **Geolocalização**: Visualização da localização de cada supermercado através de mapas interativos com Leaflet e OpenStreetMap, com geocodificação automática por nome de localidade via Nominatim.
- **Sistema de Reembolsos** *(iniciativa do grupo)*: Funcionalidade adicional não prevista no enunciado, que permite ao cliente submeter pedidos de reembolso diretamente pelo frontoffice. O supermercado pode aprovar ou rejeitar o pedido através do backoffice, tornando o fluxo pós-venda mais completo e próximo de um ambiente de produção real.
- **Cancelamento Temporal**: Janela de 5 minutos para cancelamento de encomendas confirmadas e não pagas, com contador visível ao cliente.
- **Comparação de Preços**: Agregação em tempo real do mesmo produto em diferentes supermercados, ordenada por preço.

## 4.7 Interface Visual do Sistema

Para consolidar a especificação técnica descrita ao longo deste capítulo, apresentam-se abaixo algumas imagens que ilustram a interface final do sistema desenvolvido. A componente visual foi desenhada com foco na usabilidade e responsividade.

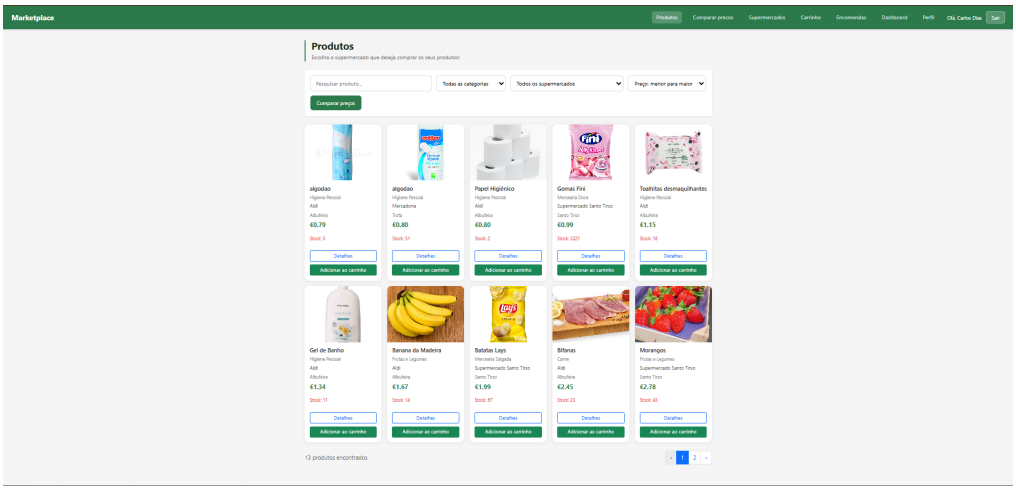



Figura 4.1: Catálogo de produtos e pesquisa no Frontoffice Angular

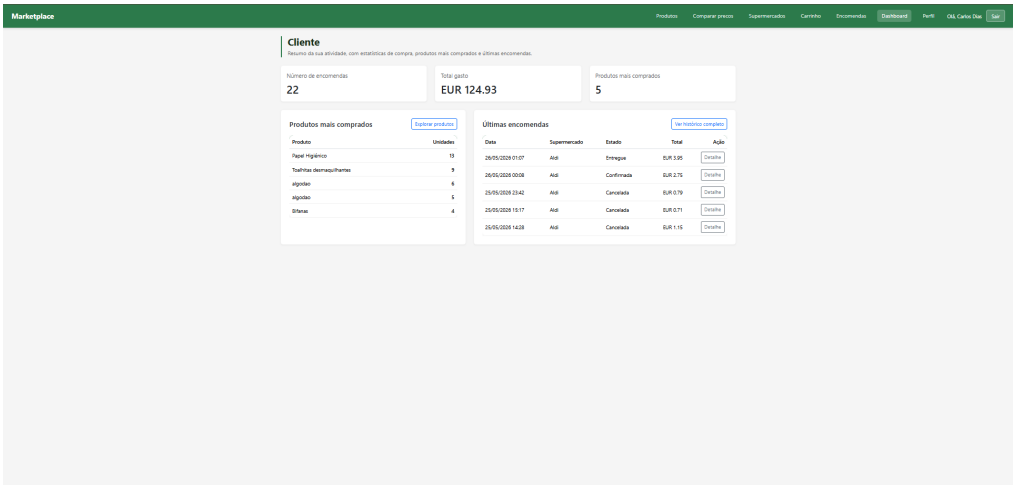



Figura 4.2: Dashboard Cliente Final

# Execução do Projeto

Para validar as funcionalidades desenvolvidas e testar a plataforma em ambiente de desenvolvimento, é necessário colocar em execução, de forma independente, tanto o servidor de suporte (Backend) como a aplicação do cliente (Frontend). Devem ser seguidos os seguintes passos técnicos:

## 1. Backend (API REST e Backoffice EJS)

1. **Instalação de Dependências:** Navegar até à raiz do projeto através do terminal e executar o comando `npm install` para descarregar e instalar todos os módulos listados no ficheiro `package.json`.
2. **Configuração de Ambiente:** Criar e parametrizar o ficheiro `.env` na raiz do backend. Devem ser definidas as variáveis estruturais do sistema, nomeadamente: `MONGODB_URI` (string de ligação ao MongoDB Atlas), `PORT` (porta de escuta do servidor), `APP_BASE_URL`, `STRIPE_SECRET_KEY` (chave privada para processamento de pagamentos), `JWT_SECRET` (chave de segurança para assinatura dos tokens de cliente) e as credenciais SMTP para o disparo de emails automáticos.
3. **Arranque do Servidor:** Iniciar a aplicação através do comando `npm start`. O servidor Express ficará ativo e as rotas de Backoffice/API ficarão disponíveis no endereço `http://localhost:3000`. A documentação Swagger poderá ser consultada interativamente em `http://localhost:3000/api-docs`.

## 2. Frontend (Aplicação Cliente Angular)

1. **Instalação do Módulo Cliente:** Abrir uma nova instância do terminal, aceder à pasta dedicada à aplicação decorativa através do comando `cd frontoffice` e executar novamente `npm install` para instalar as dependências específicas do ecossistema Angular.
2. **Arranque da Aplicação:** Executar o comando `npm start` (ou `ng serve`) dentro do diretório do frontend. O processo de compilação local será iniciado e, uma vez concluído com sucesso, o Marketplace ficará acessível para o utilizador final através do endereço `http://localhost:4200`.

**Nota de Validação:** Para a execução de um ciclo de testes completo end-to-end, recomenda-se o recurso às contas de teste pré-configuradas na base de dados (apresentadas na Secção 1.4). Isto permitirá validar as restrições de segurança impostas pelos middlewares de controlo de acessos, bem como as capacidades específicas de cada perfil de utilizador.

# Conclusão e Trabalho Futuro

## 6.1 Síntese dos Resultados Alcançados

O projeto foi desenvolvido em duas fases, ambas concluídas com sucesso dentro dos prazos estabelecidos.

No Milestone 1 foi entregue um backoffice funcional construído com Node.js, Express.js, MongoDB Atlas e EJS, cobrindo a totalidade dos requisitos obrigatórios: gestão de utilizadores com quatro perfis distintos, aprovação de supermercados pelo administrador, CRUD completo de produtos com suporte a imagens e códigos EAN, sistema de encomendas com controlo de estados, interface de venda em caixa, e integração com Stripe Checkout e Nodemailer. Foram ainda implementadas funcionalidades de bonificação como o sistema de cupões, dashboards com estatísticas e avaliações de supermercados e estafetas.

No Milestone 2 foi desenvolvido o frontoffice Angular para clientes finais, comunicando com o backend através de uma API REST autenticada por JWT e documentada com Swagger. A aplicação Angular cobre o catálogo de produtos com filtros combinados, comparação de preços entre supermercados, carrinho de compras com validação de stock em tempo real, *checkout* com seleção de método de entrega e aplicação de cupões, pagamento via Stripe, histórico de encomendas com cancelamento temporal de 5 minutos, sistema de *reviews*, dashboard do cliente e geolocalização dos supermercados com Leaflet e OpenStreetMap.

O sistema final cobre todos os requisitos obrigatórios do enunciado e implementa a totalidade das funcionalidades de bonificação propostas, incluindo notificações por email, pagamento online, sistema de avaliações, cupões de desconto e geolocalização.

## 6.2 Trabalho Futuro

Apesar do sistema se encontrar funcional e completo face aos requisitos do enunciado, existem várias melhorias que poderiam ser implementadas numa fase posterior para aproximar a plataforma de um ambiente de produção real.

A primeira melhoria seria a implementação de notificações em tempo real via WebSockets ou *Server-Sent Events*, permitindo que o cliente acompanhe a evolução do estado da sua encomenda sem necessidade de recarregar a página, e que o estafeta receba alertas imediatos quando uma nova entrega fica disponível.

Uma segunda área de melhoria seria a conversão da aplicação Angular numa *Progressive Web App* (PWA), adicionando suporte a instalação no dispositivo móvel, funcionamento *offline* para consulta do histórico, e notificações *push* nativas.

Do ponto de vista operacional, seria útil implementar um painel de *analytics* avançado para o administrador, com métricas agregadas sobre volume de encomendas por período, receita por supermercado, taxa de cancelamento e tempo médio de entrega, recorrendo a uma biblioteca de visualização como Chart.js ou D3.js.

Por fim, a introdução de um sistema de pesquisa por geolocalização permitiria ao cliente filtrar supermercados por proximidade à sua morada, calculando a distância em tempo real com base nas coordenadas armazenadas para cada loja.